

Networking Fundamentals

Unit 03 — Technical Foundations

You can't attack — or defend — a network you don't understand. This week we learn how data actually travels, from zero.

Learning objectives

By the end of this unit you can:

- **Define** a network and trace, in order, how data travels between devices.
- **Distinguish** an IP address from a MAC address.
- **Identify** an IPv4 address as public or private.
- **Match** 8+ common ports to their services.
- **Explain** TCP vs. UDP and give an example of each.

Learning objectives (cont.)

- **Diagram** the TCP three-way handshake (SYN, SYN-ACK, ACK).
- **Describe** the TCP/IP 4-layer model and relate it to OSI.
- **Trace** a DNS lookup from name to IP.
- **Capture** live traffic in Wireshark and identify protocol, IPs, and ports.
- **Locate** the three handshake packets in a real capture.



Ethics: capture is wiretapping

Capturing traffic that isn't yours can be a crime — even if you "just looked."

-  Capturing on a network you **own or are authorized to test** (this lab, your home) is fine.
-  Capturing others' traffic — public Wi-Fi, a friend's network, the school's general network — can violate wiretap and computer-crime laws.

The dividing line, as always: **authorization and scope.**

Day 1 — What is a network?

IPs and MACs

Warm-up

"How does a text message actually reach your friend's phone?"

Sketch your best guess. We'll keep these and redraw them on Day 5.

Networks, hosts & routers

Term	Meaning
Network	Two or more devices connected to share data
Host	Any device on a network (computer, phone, printer, server)
Router	A device that forwards packets between different networks

Data hops from host → router → router → host until it arrives.

IP address

- An **IP address** identifies a device on a network so traffic can find it.
- **IPv4** is written as four numbers: `192.168.1.10`.
- Think of it as a **mailing address** — it can change depending on where you are.

MAC address

- A **MAC address** is a hardware ID burned into the network card:
`00:1A:2B:3C:4D:5E`.
- Think of it as your **name on the mailbox** — fixed to the device.

	IP address	MAC address
What	Logical, where you are	Hardware, who you are
Changes?	Yes	No (fixed)
Used for	Routing across the internet	Delivery on the local segment

Public vs. private IPs

Type	Reachable from the internet?
Public	Yes — out on the open internet
Private	No — only inside a local network

Memorize the **three private ranges**:

- `10.0.0.0/8`
- `172.16.0.0/12` (that's `172.16` – `172.31`)
- `192.168.0.0/16`

Private devices reach the internet through **NAT** at the router.

Find your own addresses

```
# Linux / macOS  
ip addr          # (or: ifconfig)  
  
# Windows  
ipconfig /all
```

Record your **IPv4** and your **MAC**. Is your IP public or private — and how can you tell?

Day 1 exit ticket

"Is `192.168.0.42` public or private? How do you know?"

Day 2 — Ports, services & the models

Ports and services

- One host has **one IP** but many **ports** — numbered "doors."
- Analogy: one street address (IP), many apartment doors (ports).
- A **service** is a program that listens on a port and answers requests.
- A **protocol** is the agreed set of rules for how two devices talk.

Common ports to know

Port	Service	Encrypted?
20/21	FTP	No
22	SSH	Yes
23	Telnet	No
25	SMTP	No
53	DNS	No
80	HTTP	No
443	HTTPS	Yes
445	SMB	No (by default)

The TCP/IP model (4 layers)

The practical model the internet actually uses:

Layer	Example
Application	HTTP, DNS — the app's data
Transport	TCP, UDP — ports live here
Internet	IP — IP addresses live here
Link	Ethernet, Wi-Fi — MAC lives here

OSI vs. TCP/IP

- **OSI** is a 7-layer **reference map** (cables → apps).
- **TCP/IP** is the practical 4-layer model the internet uses.

Don't memorize all 7 OSI layers. Just answer: **which layer is this thing on?**

- IP address → Internet/Network layer
- Port → Transport layer
- HTTP → Application layer

Day 2 exit ticket

"Name the port for HTTPS, SSH, and DNS."

Day 3 — TCP vs. UDP & the three-way handshake

Warm-up

"A phone call vs. dropping a postcard in the mail — which one confirms the other side received it?"

That's the difference between **TCP** and **UDP**.

TCP vs. UDP

	TCP	UDP
Connection	Yes (handshake first)	None — just send
Reliable?	Confirms delivery, ordered	No guarantee
Speed	Slower	Faster
Use for	Web pages, SSH, downloads	DNS, video/voice streaming, games

TCP = the phone call. **UDP** = the postcard.

The three-way handshake

TCP opens every connection with **three** packets:

```
Client  --- SYN -----> Server      (1) "Can we talk?"
Client  <-- SYN-ACK ---- Server      (2) "Yes – can you?"
Client  --- ACK -----> Server      (3) "Yes. Connected."
```

SYN → SYN-ACK → ACK. Then data flows.

Watch the direction

- There is **only one SYN** — from the client.
- The **SYN-ACK is a single packet** sent **back from the server**.
- Then the client sends the final **ACK**.

Common mistake: expecting two SYNs. The server's reply combines SYN **and** ACK into one packet.

Day 3 exit ticket

"Put SYN, ACK, SYN-ACK in the right order."

Then diagram the handshake from memory and label each packet's flags.

Day 4 — DNS, packets, firewalls & a first capture

Warm-up

"You type `example.com`. Your computer doesn't know what that means yet. What has to happen first?"

DNS — the internet's phonebook

DNS turns a name like `example.com` into an **IP address**.

You type `example.com`

- your computer asks a recursive resolver
- resolver asks root → TLD (.com) → authoritative server
- answer comes back: `93.184.x.x`
- your browser connects to that IP

DNS queries usually use **UDP port 53**; the response carries an **A record** (IPv4).

Anatomy of a packet

A **packet** is a small chunk of data wrapped with addressing info.

Part	Holds
Headers	Source/dest IP, ports, protocol, flags
Payload	The actual data being carried

Routers read the headers to forward each packet toward its destination.

Firewalls

A **firewall** is a filter that **allows or blocks** traffic based on rules.

- Rules can match by **port**, **IP**, or **protocol**.
- Example: "allow port 443 in, block port 23."
- The first thing many attacks run into.

Wireshark & tcpdump

Tool	What it is
Wireshark	A graphical tool to capture and inspect packets
tcpdump	A command-line packet-capture tool



Reminder: capture **only** on networks you own or are authorized to test.

Day 4 exit ticket

"What does a DNS query ask for, and what does it get back?"

Then begin the Wireshark capture lab — your first capture of a `ping`.

Day 5 — Capture lab + putting it together



Lab safety & authorization

reminder

You may only run these techniques inside this lab environment. Capture **only** traffic on the classroom lab network or your own device's loopback/test traffic, exactly as your instructor directs. **Do not** run Wireshark or tcpdump on the school's general network, public Wi-Fi, or anyone else's network — intercepting others' communications can be a crime (wiretapping) **even if you never use what you see**. Authorization and scope are the line.

Restate this in your own words at the top of your journal.

Display filters you'll use

In the Wireshark filter bar:

```
icmp          # show only ping traffic
dns           # show only DNS
tcp.port == 80 # show only HTTP TCP traffic
ip.addr == 93.184.x.x # show traffic to/from one IP
```

Filters don't change what you captured — they change what you **see**.

Lab Step 1 — Finish the TryHackMe room

Work through the assigned networking room. In your own words, journal:

- What an IP address is; public vs. private.
- What a MAC address is and how it differs from an IP.
- Three port→service pairs (e.g., 22/SSH).
- The difference between TCP and UDP.



Screenshot the completed-room banner.

Lab Step 2 — Find your addresses

```
# Linux / macOS  
ip addr  
  
# Windows  
ipconfig /all
```

Record your **IPv4** and **MAC**. Note whether the IP is **public or private** and how you can tell.

Lab Step 3 — Capture a ping

1. In Wireshark, double-click your **assigned** interface to start capturing.
2. Ping a target your instructor approves:

```
ping -c 4 8.8.8.8      # Linux/macOS  
ping -n 4 8.8.8.8      # Windows
```

3. Stop the capture. Filter: `icmp`.

You'll see **Echo (ping) request** / **reply** pairs. Record which IP asked and which answered.

Note: ICMP is **not** TCP or UDP — it has **no ports**.

37

Lab Step 4 — Capture a web load & DNS

1. Start a new capture.
2. Load an approved page — use **http** so traffic is visible (`http://example.com`).
HTTPS would be encrypted.
3. Stop the capture. Filter: `dns`.

You'll see a DNS **query** for the domain and a **response** with one or more IPs. Record: **name looked up → IP returned**. That's DNS resolution, live.

Lab Step 5 — Find the handshake

Filter to the web server's traffic:

```
ip.addr == <the IP DNS returned>  
# or:  tcp.port == 80
```

Find the first **three** TCP packets (check the **Info** column):

- Packet 1: **[SYN]** — your machine asking to connect
- Packet 2: **[SYN, ACK]** — the server agreeing
- Packet 3: **[ACK]** — your machine confirming → connected

Record the source IP, dest IP, and **port** of each.

Lab Step 6 — Annotate

For **three** packets (at least one from the handshake), fill in your observation table:

Packet #	Protocol	Source IP	Dest IP	Port(s)	What it's doing
Server side = well-known port (80/443). Client side = a random high "ephemeral" port (like 50112).					

Reading a packet (worked example)

Source: 192.168.1.23:50112 Dest: 93.184.216.34:80 Flags: [SYN]

- **Protocol:** TCP
- **Client:** 192.168.1.23:50112 (private IP, high ephemeral port)
- **Server:** 93.184.216.34:80 (port 80 = HTTP)
- **Doing:** the **first** packet of the three-way handshake — client asking to open a connection. Next expected: [SYN, ACK] from the server.

Lab deliverables

Submit your lab-journal page with:

- The safety reminder in your own words.
- Screenshot of the completed TryHackMe room.
- Your device's IP + MAC, labeled public/private.
- The annotated table (≥ 3 packets, ≥ 1 handshake packet).
- The DNS query/response (name $\rightarrow$ IP).
- A 3–5 sentence "**packet story**" + one surprise or open question.

Full vocabulary (1 of 2)

Term	Meaning
Network / Host	Connected devices / any device on one
IP address / IPv4	Numeric device address / four-number format
Public / Private IP	Reachable on the internet / local-only
MAC address	Fixed hardware ID on the network card
Port / Service	Numbered "door" / program listening on it
Protocol	Agreed rules for how devices talk
Router / Firewall	Forwards between networks / allows-blocks by rule

Full vocabulary (2 of 2)

Term	Meaning
TCP	Connection-based, reliable, confirms delivery
UDP	Connectionless, fast, no guarantee
Three-way handshake	SYN → SYN-ACK → ACK starts a TCP connection
OSI model	7-layer reference map
TCP/IP model	Practical 4-layer model the internet uses
Packet	A chunk of data wrapped with addressing info
DNS	Turns names into IP addresses
Wireshark / tcpdump	Graphical / command-line packet capture

Recap

- A **network** connects hosts; **routers** move packets between networks.
- **IP** = logical, can change; **MAC** = fixed hardware. Private ranges: `10`, `172.16-31`, `192.168`.
- **Ports** are doors; know the common ones (22 SSH, 80 HTTP, 443 HTTPS, 53 DNS).
- **TCP** confirms delivery (3-way handshake); **UDP** just sends.
- **DNS** turns names into IPs. **Firewalls** allow/block by rule.
- **Wireshark** reads packets — only where you're **authorized**.

Exit ticket & discussion

1. List the **three private IPv4 ranges**. Why can't they be reached directly from the internet?
2. Trace `example.com` → page loading, using **DNS, IP, three-way handshake, port**.
3. In a TCP web request, why is the **client's** port a random high number while the **server's** is well-known?

Discuss: Running Wireshark at a coffee shop and seeing others' traffic — legal? ethical? Where's the line between curiosity and interception?

Submit: your annotated capture lab-journal page + packet story.